



TUTORIAL: Applicazione E-Commerce Completa in Kotlin & Jetpack Compose

Questo progetto realizza l'applicazione di catalogo e carrello spesa in modalità **Android Nativa**. Utilizza **Jetpack Compose** per l'interfaccia grafica descrittiva, **Retrofit** per le chiamate di rete HTTP e un **ViewModel** con lo stato integrato per la gestione globale del carrello.



Fase 0: Creazione del Progetto da Zero (Android Studio)

Jetpack Compose è lo standard nativo di Android, quindi questa procedura si esegue esclusivamente all'interno di **Android Studio**.

1. Apri **Android Studio**.
2. Nella schermata di benvenuto, clicca su **New Project**.
3. Nella finestra dei modelli, seleziona **Empty Activity** (è il modello ufficiale che configura già Jetpack Compose, Kotlin e tutte le dipendenze grafiche di base). Clicca su **Next**.
4. Configura i dettagli del progetto compilando i campi in questo modo:
 - **Name:** `OnlineShopAppKotlin`
 - **Package name:** `com.example.onlineshopappkotlin`
 - **Language:** Kotlin (*bloccato di default*)
 - **Minimum SDK:** API 24 (Android 7.0 Nougat) o superiore per garantire la massima compatibilità.
5. Clicca su **Finish**. Android Studio scaricherà Gradle e configurerà l'ambiente.

2. Gestione delle Dipendenze (`build.gradle.kts`)

Per fare le chiamate di rete e caricare le immagini da internet, dobbiamo aggiungere le librerie industriali standard per Android: **Retrofit** (per le API) e **Coil** (per caricare le immagini via URL).

Apri il file `app/build.gradle.kts` (Module: app) e aggiungi all'interno del blocco `dependencies { ... }` le seguenti righe:

Kotlin

```
dependencies {  
    // Retrofit per le chiamate HTTP e la conversione JSON automatica (GSON)  
    implementation("com.squareup.retrofit2:retrofit:2.9.0")  
}
```

```

implementation("com.squareup.retrofit2:converter-gson:2.9.0")

// Coil per il caricamento asincrono delle immagini tramite URL
implementation("io.coil-kt:coil-compose:2.4.0")

// Ciclo di vita e integrazione ViewModel con Compose
implementation("androidx.lifecycle:lifecycle-viewmodel-compose:2.6.1")
}

```

Clicca su **Sync Now** in alto a destra per permettere ad Android Studio di scaricare i pacchetti.



Permesso Internet Obbligatorio

Dato che l'app scarica i prodotti da un server online, devi dare il permesso ad Android di accedere a internet. Apri il file `app/src/main/AndroidManifest.xml` e aggiungi questa riga subito sopra il blocco `<application>`:

XML

```
<uses-permission android:name="android.permission.INTERNET" />
```

3. Struttura dei File e Codice Sorgente Blindato

Crea i file Kotlin all'interno del tuo package principale (`app/src/main/java/com/example/online-shop-app-kotlin/`). Crea le stesse cartelle logiche per rispettare l'architettura pulita:

Plaintext

com.example.online-shop-app-kotlin/

```

|
|— MainActivity.kt
|
|— models/
|   |— Product.kt
|
|— services/
|   |— ApiService.kt
|
|— viewmodels/
|   |— CartViewModel.kt
|
|— screens/
|   |— CatalogScreen.kt
|   |— DetailScreen.kt
|   |— CartScreen.kt

```

1. Il Modello Dati (**models/Product.kt**)

*Mappatura degli oggetti in arrivo dal JSON. In Kotlin usiamo le **data class**, perfette per memorizzare dati in modo pulito.*

Kotlin

```
package com.example.onlineshopappkotlin.models
```

```
// Modello per la Categoria del prodotto
```

```
data class Category(  
    val id: Int,  
    val name: String,  
    val image: String  
)
```

```
// Modello principale per il Prodotto
```

```
data class Product(  
    val id: Int,  
    val title: String,  
    val price: Double,  
    val description: String,  
    val images: List<String>,  
    val category: Category  
)
```

```
// Modello per l'elemento all'interno del Carrello (Prodotto + Quantità modificabile)
```

```
data class CartItem(  
    val product: Product,  
    var quantity: Int = 1  
)
```

2. Il Servizio API (**services/ApiService.kt**)

In Android Nativo, Retrofit mappa l'endpoint tramite un'interfaccia. Creiamo l'interfaccia e l'oggetto Singleton che istanzia il client HTTP.

Kotlin

```
package com.example.onlineshopappkotlin.services
```

```
import com.example.onlineshopappkotlin.models.Category  
import com.example.onlineshopappkotlin.models.Product  
import retrofit2.Retrofit  
import retrofit2.converter.gson.GsonConverterFactory  
import retrofit2.http.GET  
import retrofit2.http.Path
```

```
// Interfaccia che definisce gli endpoint dell'API di Platzi
interface ApiInterface {
    @GET("products?offset=0&limit=30")
    suspend fun getProducts(): List<Product>

    @GET("products/{id}")
    suspend fun getProductDetails(@Path("id") id: Int): Product
}

// Singleton per accedere al servizio di rete ovunque nell'app
object ApiService {
    private const val BASE_URL = "https://api.escuelajs.co/api/v1/"

    val client: ApiInterface by lazy {
        Retrofit.Builder()
            .baseUrl(BASE_URL)
            .addConverterFactory(GsonConverterFactory.create()) // Mappa automaticamente il
JSON in oggetti Kotlin
            .build()
            .create(ApiInterface::class.java)
    }
}
```

3. Gestione dello Stato Globale (**viewmodels/CartViewModel.kt**)

*Il ViewModel sostituisce il Provider di Flutter. Sopravvive ai cambi di orientamento dello schermo e distribuisce lo stato del carrello in modo reattivo usando **mutableStateListOf**.*

Kotlin

```
package com.example.onlineshopappkotlin.viewmodels
```

```
import androidx.compose.runtime.mutableStateListOf
import androidx.lifecycle.ViewModel
import com.example.onlineshopappkotlin.models.CartItem
import com.example.onlineshopappkotlin.models.Product
```

```
class CartViewModel : ViewModel() {
    // Lista osservabile dall'interfaccia grafica Compose
    private val _cartItems = mutableStateListOf<CartItem>()
    val cartItems: List<CartItem> get() = _cartItems

    // Aggiunge un prodotto o ne aumenta la quantità
    fun addToCart(product: Product) {
        val existingItem = _cartItems.find { it.product.id == product.id }
        if (existingItem != null) {
```

```

        val index = _cartItems.indexOf(existingItem)
        _cartItems[index] = existingItem.copy(quantity = existingItem.quantity + 1)
    } else {
        _cartItems.add(CartItem(product = product, quantity = 1))
    }
}

// Riduce la quantità o rimuove l'elemento se scende a zero
fun removeFromCart(productId: Int) {
    val existingItem = _cartItems.find { it.product.id == productId }
    if (existingItem != null) {
        val index = _cartItems.indexOf(existingItem)
        if (existingItem.quantity == 1) {
            _cartItems.removeAt(index)
        } else {
            _cartItems[index] = existingItem.copy(quantity = existingItem.quantity - 1)
        }
    }
}

// Svuota tutto il carrello
fun clearCart() {
    _cartItems.clear()
}

// Calcolo reattivo del totale dei pezzi
val totalItemsCount: Int
    get() = _cartItems.sumOf { it.quantity }

// Calcolo reattivo del prezzo totale complessivo
val totalPrice: Double
    get() = _cartItems.sumOf { it.product.price * it.quantity }
}

```

4. Schermata Catalogo (**screens/CatalogScreen.kt**)

Equivalente del CatalogScreen di Flutter. Gestisce il caricamento asincrono dei dati ed effettua il medesimo filtro di pulizia delle URL.

Kotlin

```
package com.example.onlineshopappkotlin.screens
```

```

import androidx.compose.foundation.clickable
import androidx.compose.foundation.layout.*
import androidx.compose.foundation.lazy.LazyColumn
import androidx.compose.foundation.lazy.items
import androidx.compose.foundation.shape.RoundedCornerShape

```

```

import androidx.compose.material3.*
import androidx.compose.runtime.*
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.draw.clip
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.layout.ContentScale
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp
import coil.compose.AsyncImage
import com.example.onlineshopappkotlin.models.Product
import com.example.onlineshopappkotlin.services.ApiService
import com.example.onlineshopappkotlin.viewmodels.CartViewModel

// Pulisce le URL delle immagini ereditate dall'API pubblica
fun cleanImageUrl(url: String): String {
    val cleaned = url.replace("[", "").replace("]", "").replace("\\", "")
    return if (!cleaned.startsWith("http")) "https://via.placeholder.com/150" else cleaned
}

@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun CatalogScreen(
    viewModel: CartViewModel,
    onNavigateToDetail: (Int) -> Unit,
    onNavigateToCart: () -> Unit
) {
    // Stati per gestire la chiamata di rete asincrona
    var products by remember { mutableStateOf<List<Product>>(emptyList()) }
    var isLoading by remember { mutableStateOf(true) }
    var isError by remember { mutableStateOf(false) }

    // Carica i dati all'avvio del componente grafico (simile all'initState)
    LaunchedEffect(Unit) {
        try {
            val rawProducts = ApiService.client.getProducts()
            // FILTRO ANTI-CRASH IDENTICO A FLUTTER: Pulisce i dati corrotti a monte
            products = rawProducts.filter { product ->
                product.id != 0 &&
                product.title.isNotEmpty() &&
                product.price > 0 &&
                product.images.isNotEmpty() &&
                product.images[0].startsWith("http")
            }
            isLoading = false
        } catch (e: Exception) {
            isError = true
        }
    }
}

```

```

        isLoading = false
    }
}

Scaffold(
  topBar = { TopAppBar(title = { Text("Catalogo Prodotti") }) }
) { paddingValues ->
  Box(
    modifier = Modifier
      .fillMaxSize()
      .padding(paddingValues)
  ) {
    if (isLoading) {
      CircularProgressIndicator(
        modifier = Modifier.align(Alignment.Center),
        color = Color(0xFF4F46E5)
      )
    } else if (isError) {
      Text(
        text = "⚠ Errore nel caricamento dei dati",
        color = Color.Red,
        modifier = Modifier.align(Alignment.Center)
      )
    } else {
      Column(modifier = Modifier.fillMaxSize()) {
        // Lista a scorrimento verticale (Equivalente di ListView.builder)
        LazyColumn(
          modifier = Modifier
            .weight(1f)
            .padding(horizontal = 16.dp)
        ) {
          items(products) { product ->
            Card(
              modifier = Modifier
                .fillMaxWidth()
                .padding(vertical = 8.dp)
                .clickable { onNavigateToDetail(product.id) },
              shape = RoundedRectangleBorder(12.dp),
              colors = CardDefaults.cardColors(containerColor = Color.White),
              elevation = CardDefaults.cardElevation(2.dp)
            ) {
              Row(modifier = Modifier.padding(16.dp), verticalAlignment =
Alignment.CenterVertically) {
                // AsyncImage di Coil sostituisce Image.network di Flutter
                AsyncImage(
                  model = cleanImageUrl(product.images[0]),
                  contentDescription = null,
                  modifier = Modifier

```

```

        .size(60.dp)
        .clip(RoundedCornerShape(8.dp)),
        contentScale = ContentScale.Cover
    )
    Spacer(modifier = Modifier.width(16.dp))
    Column(modifier = Modifier.weight(1f)) {
        Text(text = product.title, fontWeight = FontWeight.Bold, fontSize =
16.sp)
        Text(text = product.category.name.uppercase(), fontSize = 11.sp,
color = Color.Gray)
        Spacer(modifier = Modifier.height(4.dp))
        Text(text = "€ ${String.format("%.2f", product.price)}", color =
Color(0xFF4F46E5), fontWeight = FontWeight.bold)
    }
}
}
}
}

// Barra inferiore fissa del Carrello
Surface(
    modifier = Modifier.fillMaxWidth(),
    shadowElevation = 8.dp,
    color = Color.White
){
    Row(
        modifier = Modifier
            .fillMaxWidth()
            .padding(16.dp),
        horizontalArrangement = Arrangement.SpaceBetween, // Sostituisce
MainAxisAlignment.spaceBetween
        verticalAlignment = Alignment.CenterVertically
    ){
        Column {
            Text(text = "${viewModel.totalItemsCount} prodotti nel carrello",
fontWeight = FontWeight.Bold)
            Text(text = "Totale stimato: € ${String.format("%.2f",
viewModel.totalPrice)}", fontSize = 12.sp, color = Color.Gray)
        }
        Button(
            onClick = onNavigateToCart,
            colors = ButtonDefaults.buttonColors(containerColor =
Color(0xFF4F46E5))
        ){
            Text("Vai al carrello", color = Color.White)
        }
    }
}
}

```



```

    }
  }
}
}
}

```

5. Schermata Dettaglio (**screens/DetailScreen.kt**)

Mostra la scheda del singolo prodotto selezionato tramite ID e permette l'aggiunta dell'elemento al carrello.

Kotlin

```
package com.example.onlineshopappkotlin.screens
```

```

import androidx.compose.foundation.layout.*
import androidx.compose.foundation.rememberScrollState
import androidx.compose.foundation.shape.RoundedCornerShape
import androidx.compose.foundation.verticalScroll
import androidx.compose.material3.*
import androidx.compose.runtime.*
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.layout.ContentScale
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp
import coil.compose.AsyncImage
import com.example.onlineshopappkotlin.models.Product
import com.example.onlineshopappkotlin.services.ApiService
import com.example.onlineshopappkotlin.viewmodels.CartViewModel
import kotlinx.coroutines.launch

```

```

@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun DetailScreen(
    productId: Int,
    viewModel: CartViewModel,
    onNavigateBack: () -> Unit
) {
    var product by remember { mutableStateOf<Product?>(null) }
    var isLoading by remember { mutableStateOf(true) }
    val scope = rememberCoroutineScope()
    val snackbarHostState = remember { SnackbarHostState() }

    LaunchedEffect(productId) {
        try {

```

```

        product = ApiService.client.getProductDetails(productId)
        isLoading = false
    } catch (e: Exception) {
        isLoading = false
    }
}

Scaffold(
    topBar = { TopAppBar(title = { Text("Dettaglio Prodotto") }) },
    snackbarHost = { SnackbarHost(snackbarHostState) }
) { paddingValues ->
    Box(
        modifier = Modifier
            .fillMaxSize()
            .padding(paddingValues)
    ) {
        if (isLoading) {
            CircularProgressIndicator(modifier = Modifier.align(Alignment.Center), color =
Color(0xFF4F46E5))
        } else {
            product?.let { currentProduct ->
                Column(
                    modifier = Modifier
                        .fillMaxSize()
                        .verticalScroll(rememberScrollState()) // Rende la scheda scorrevole
verticalmente
                ) {
                    AsyncImage(
                        model = cleanImageUrl(currentProduct.images[0]),
                        contentDescription = null,
                        modifier = Modifier
                            .fillMaxWidth()
                            .height(300.dp),
                        contentScale = ContentScale.Cover
                    )
                    Column(modifier = Modifier.padding(20.dp)) {
                        Text(text = currentProduct.category.name.uppercase(), color = Color.Gray,
fontWeight = FontWeight.Bold, fontSize = 12.sp)
                        Spacer(modifier = Modifier.height(8.dp))
                        Text(text = currentProduct.title, fontSize = 22.sp, fontWeight =
FontWeight.bold)
                        Spacer(modifier = Modifier.height(8.dp))
                        Text(text = "€ ${String.format("%.2f", currentProduct.price)}", fontSize =
20.sp, color = Color(0xFF4F46E5), fontWeight = FontWeight.bold)
                        Spacer(modifier = Modifier.height(16.dp))
                        Text(text = currentProduct.description, fontSize = 15.sp, color =
Color.DarkGray, lineHeight = 20.sp)
                        Spacer(modifier = Modifier.height(30.dp))
                    }
                }
            }
        }
    }
}

```

al carrello!")

Gestisce la manipolazione degli elementi del carrello e mostra un Dialog nativo per confermare l'ordine pulendo lo stato globale.

Kotlin

```
package com.example.onlineshopappkotlin.screens
```

```
import androidx.compose.foundation.layout.*
import androidx.compose.foundation.lazy.LazyColumn
import androidx.compose.foundation.lazy.items
import androidx.compose.foundation.shape.RoundedCornerShape
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.outlined.Delete
import androidx.compose.material3.*
```

```

import androidx.compose.runtime.*
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.res.painterResource
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp
import com.example.onlineshopappkotlin.viewmodels.CartViewModel

@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun CartScreen(
    viewModel: CartViewModel,
    onNavigateBack: () -> Unit
) {
    var showDialog by remember { mutableStateOf(false) }

    if (showDialog) {
        AlertDialog(
            onDismissRequest = { showDialog = false },
            title = { Text("Ordine Confermato!") },
            text = { Text("Simulazione d'acquisto completata con successo.") },
            confirmButton = {
                TextButton(onClick = {
                    viewModel.clearCart()
                    showDialog = false
                    onNavigateBack() // Torna alla home dopo l'acquisto
                }) { Text("OK") }
            }
        )
    }

    Scaffold(
        topBar = { TopAppBar(title = { Text("Il mio Carrello") }) }
    ) { paddingValues ->
        Column(
            modifier = Modifier
                .fillMaxSize()
                .padding(paddingValues)
        ) {
            if (viewModel.cartItems.isEmpty()) {
                Box(modifier = Modifier.weight(1f).fillMaxWidth(), contentAlignment =
Alignment.Center) {
                    Text("Il carrello è vuoto.", color = Color.Gray, fontSize = 16.sp)
                }
            } else {
                LazyColumn(

```

```

        modifier = Modifier
            .weight(1f)
            .padding(horizontal = 16.dp)
    ){
        items(viewModel.cartItems) { item ->
            Card(
                modifier = Modifier.fillMaxWidth().padding(vertical = 6.dp),
                shape = RoundedCornerShape(12.dp),
                colors = CardDefaults.cardColors(containerColor = Color.White)
            ){
                Row(modifier = Modifier.padding(16.dp), verticalAlignment =
Alignment.CenterVertically) {
                    Column(modifier = Modifier.weight(1f)) {
                        Text(text = item.product.title, fontWeight = FontWeight.Bold, fontSize
= 16.sp)

                        Text(text = "Unitario: € ${String.format("%.2f", item.product.price)}",
color = Color.Gray, fontSize = 12.sp)
                        Spacer(modifier = Modifier.height(4.dp))
                        Text(text = "Subtotale: € ${String.format("%.2f", item.product.price *
item.quantity)}", color = Color(0xFF4F46E5), fontWeight = FontWeight.bold)
                    }
                    Row(verticalAlignment = Alignment.CenterVertically) {
                        IconButton(onClick = { viewModel.removeFromCart(item.product.id) })
{

                            Text("-", fontSize = 20.sp, fontWeight = FontWeight.Bold)
                        }
                        Text(text = "${item.quantity}", fontSize = 16.sp, fontWeight =
FontWeight.Bold)

                        IconButton(onClick = { viewModel.addToCart(item.product) }) {
                            Text("+", fontSize = 20.sp, fontWeight = FontWeight.Bold)
                        }
                    }
                }
            }
        }
    }

    // Area dei Totali e pulsanti d'azione
    Surface(modifier = Modifier.fillMaxWidth(), shadowElevation = 8.dp, color =
Color.White) {
        Column(modifier = Modifier.padding(20.dp)) {
            Row(modifier = Modifier.fillMaxWidth(), horizontalArrangement =
Arrangement.SpaceBetween) {
                Text("Numero totale prodotti:")
                Text("${viewModel.totalItemsCount}", fontWeight = FontWeight.Bold)
            }
            Spacer(modifier = Modifier.height(8.dp))

```



```

import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.activity.viewModels
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.Surface
import androidx.lifecycle.viewmodel.compose.viewModel
import androidx.navigation.NavType
import androidx.navigation.compose.NavHost
import androidx.navigation.compose.composable
import androidx.navigation.compose.rememberNavController
import androidx.navigation.navArgument
import com.example.onlineshopappkotlin.screens.CartScreen
import com.example.onlineshopappkotlin.screens.CatalogScreen
import com.example.onlineshopappkotlin.screens.DetailScreen
import com.example.onlineshopappkotlin.viewmodels.CartViewModel

```

```

class MainActivity : ComponentActivity() {

```

```

    override fun onCreate(savedInstanceState: Bundle?) {

```

```

        super.onCreate(savedInstanceState)

```

```

        setContent {

```

```

            MaterialTheme {

```

```

                Surface(color = MaterialTheme.colorScheme.background) {

```

```

                    // Inizializza il controller di navigazione e il ViewModel centralizzato

```

```

                    val navController = rememberNavController()

```

```

                    val cartViewModel: CartViewModel = viewModel()

```

```

                    // NavHost gestisce gli spostamenti fra le pagine (Routes)

```

```

                    NavHost(navController = navController, startDestination = "catalog") {

```

```

                        // Rotta 1: Il Catalogo Principale

```

```

                        composable("catalog") {

```

```

                            CatalogScreen(

```

```

                                viewModel = cartViewModel,

```

```

                                onNavigateToDetail = { id -> navController.navigate("detail/$id") },

```

```

                                onNavigateToCart = { navController.navigate("cart") }

```

```

                            )

```

```

                        }

```

```

                    // Rotta 2: Dettaglio del Prodotto (accetta l'ID come parametro dinamico)

```

```

                    composable(

```

```

                        route = "detail/{productId}",

```

```

                        arguments = listOf(navArgument("productId") { type = NavType.IntType })

```

```

                    ) { backStackEntry ->

```

```

                        val id = backStackEntry.arguments?.getInt("productId") ?: 0

```

```
        DetailScreen(
            productId = id,
            viewModel = cartViewModel,
            onBackPressed = { navController.popBackStack() }
        )
    }

// Rotta 3: Il Carrello
composable("cart") {
    CartScreen(
        viewModel = cartViewModel,
        onBackPressed = { navController.popBackStack() }
    )
}
}
}
}
}
```

- **Gestione della Concorrenza Sicura:** In Kotlin, le chiamate di rete non possono girare sul thread principale altrimenti l'app si blocca (`NetworkOnMainThreadException`). Questo codice usa i blocchi asincroni nativi `LaunchedEffect` e funzioni `suspend` che si occupano da sole di spostare il lavoro sui thread corretti.
- **Mappatura GSON Integrata:** Non devi decodificare a mano le stringhe, Retrofit converte il JSON direttamente nelle strutture dati tramite il `GsonConverterFactory`.
- **ViewModel Architetturale Standard:** Questo rispetta le attuali linee guida ufficiali di Google (Architecture Components), un punto cruciale che i professori di ingegneria o informatica controllano subito.

Testare l'app Usare l'Emulatore Virtuale (Se il PC è potente)

1. **Apri il Device Manager:** In Android Studio, clicca sull'icona del telefono con il robotino in alto a destra (oppure vai su *Tools -> Device Manager*).
2. **Crea un emulatore:** Clicca su *Create Device*, scegli un modello (es. *Pixel 7*), clicca *Next*, scarica una versione di Android recente (es. *API 33*) e clicca *Finish*.
3. **Avvia l'emulatore:** Clicca sul tasto **Play grigio** accanto all'emulatore appena creato per accenderlo.
4. **Avvia l'app:** Una volta acceso il telefono virtuale, selezionalo nel menu in alto e clicca sul **pulsante Play Verde**.